

ElGamal Encryption System

Information Security Lecture Notes

Instructor: Sabgaht Ullah Khan

BS Computer Science

November 16, 2025

Contents

1	Introduction to ElGamal Encryption	3
1.1	What is ElGamal?	3
1.2	Historical Context	3
1.3	Why Study ElGamal?	3
2	Mathematical Foundation	4
2.1	The Discrete Logarithm Problem (DLP)	4
2.2	Primitive Roots (Generators)	4
2.3	Modular Exponentiation	5
2.4	Modular Inverse	5
3	ElGamal Encryption Algorithm	6
3.1	Overview of the Three Phases	6
3.2	Step 1: Key Generation	6
3.3	Step 2: Encryption Process	7
3.4	Step 3: Decryption Process	9
	3.4.1 Why Decryption Works (Mathematical Proof)	10
4	Complete Numerical Example	11
5	Python Implementation	13
5.1	Complete ElGamal Implementation	13
5.2	Example Output	18
6	Key Features of ElGamal	19
6.1	Advantages	19
6.2	Disadvantages	20
6.3	Homomorphic Property Demonstration	20
7	Security Considerations	21
7.1	Security Foundation	21
7.2	Critical Security Rules	22
7.3	Attack: Reusing Random Value k	22
7.4	Chosen Ciphertext Attacks	24

8	Comparison: ElGamal vs RSA	24
8.1	Comprehensive Comparison Table	24
8.2	Time Complexity Analysis	24
8.3	When to Use Each	25
9	Practical Applications	25
9.1	Real-World Usage	25
9.1.1	1. Digital Signatures (ElGamal Signature Scheme)	25
9.1.2	2. Hybrid Encryption Systems	25
9.1.3	3. Secure Voting Systems	26
9.1.4	4. Sealed-Bid Auctions	26
9.2	Implementation in Python (Production-Ready)	26
10	Summary and Key Takeaways	27
10.1	Core Concepts Review	27
10.2	Mathematical Formulas Summary	28
10.3	Security Principles	28
11	Practice Questions	28
11.1	Question 1: Basic Calculation	28
11.2	Question 2: Implementation Challenge	29
11.3	Question 3: Security Analysis	29

1 Introduction to ElGamal Encryption

1.1 What is ElGamal?

ElGamal encryption is a **public-key cryptosystem** based on the **Diffie-Hellman key exchange** principle. It was proposed by **Taher ElGamal** in 1985 and provides semantic security against chosen-plaintext attacks.

Key Characteristics

- **Type:** Asymmetric (public-key) cryptosystem
- **Security Basis:** Discrete Logarithm Problem (DLP)
- **Inventor:** Taher ElGamal (1985)
- **Property:** Probabilistic encryption (semantic security)
- **Patent Status:** Never patented (free to use)

1.2 Historical Context

The development of ElGamal was influenced by earlier work in public-key cryptography:

1. **1976:** Diffie-Hellman key exchange published
2. **1977:** RSA algorithm introduced
3. **1985:** Taher ElGamal proposes ElGamal cryptosystem
4. **Applications:** Digital signatures, hybrid encryption, cryptocurrency

1.3 Why Study ElGamal?

Advantages over RSA:

- ✓ Provides **semantic security** (probabilistic encryption)
- ✓ Same message encrypts to different ciphertexts
- ✓ No patent restrictions
- ✓ Different mathematical foundation (discrete logarithm)
- ✓ Homomorphic properties for certain operations

Trade-offs:

- × Ciphertext is twice the size of plaintext
- × Requires fresh randomness for each encryption
- × Generally slower than RSA

2 Mathematical Foundation

2.1 The Discrete Logarithm Problem (DLP)

The security of ElGamal relies on the computational hardness of the discrete logarithm problem.

Discrete Logarithm Problem

Given: A prime number p , a generator g , and a value y

Find: An integer x such that:

$$g^x \equiv y \pmod{p} \quad (1)$$

Difficulty: No known polynomial-time algorithm exists for large primes

Best known attack: Generic algorithms like Baby-step Giant-step or Pollard's rho with complexity $O(\sqrt{p})$

2.2 Primitive Roots (Generators)

A number g is a **primitive root modulo p** if its powers generate all non-zero elements modulo p .

[Primitive Root] An integer g is a primitive root modulo prime p if:

$$\{g^1 \bmod p, g^2 \bmod p, \dots, g^{p-1} \bmod p\} = \{1, 2, \dots, p-1\} \quad (2)$$

Equivalently, g has **order** $p-1$ in the multiplicative group $\mathbb{Z}_p^*$.

Example: Finding a Primitive Root

For prime $p = 11$, let's check if $g = 2$ is a primitive root:

$$\begin{aligned} 2^1 &\equiv 2 \pmod{11} \\ 2^2 &\equiv 4 \pmod{11} \\ 2^3 &\equiv 8 \pmod{11} \\ 2^4 &\equiv 16 \equiv 5 \pmod{11} \\ 2^5 &\equiv 10 \pmod{11} \\ 2^6 &\equiv 20 \equiv 9 \pmod{11} \\ 2^7 &\equiv 18 \equiv 7 \pmod{11} \\ 2^8 &\equiv 14 \equiv 3 \pmod{11} \\ 2^9 &\equiv 6 \pmod{11} \\ 2^{10} &\equiv 12 \equiv 1 \pmod{11} \end{aligned}$$

The set $\{2, 4, 8, 5, 10, 9, 7, 3, 6, 1\}$ contains all elements from 1 to 10, so $g = 2$ is a primitive root modulo 11. ✓

2.3 Modular Exponentiation

Efficient computation of $a^b \bmod m$ is crucial for ElGamal.

Algorithm 1 Fast Modular Exponentiation (Binary Method)

Require: Base a , exponent b , modulus m

Ensure: $result = a^b \bmod m$

```

1:  $result \leftarrow 1$ 
2:  $a \leftarrow a \bmod m$ 
3: while  $b > 0$  do
4:   if  $b$  is odd then
5:      $result \leftarrow (result \times a) \bmod m$ 
6:   end if
7:    $b \leftarrow \lfloor b/2 \rfloor$  ▷ Right shift
8:    $a \leftarrow (a \times a) \bmod m$  ▷ Square
9: end while
10: return  $result$ 

```

Complexity: $O(\log b)$ multiplications (very efficient!)

Example: Computing $5^6 \bmod 23$

Binary representation of 6: 110_2

$$\begin{aligned}
 5^6 &= 5^{(4+2)} = 5^4 \times 5^2 \\
 5^2 &= 25 \equiv 2 \pmod{23} \\
 5^4 &= (5^2)^2 = 2^2 = 4 \pmod{23} \\
 5^6 &= 5^4 \times 5^2 = 4 \times 2 = 8 \pmod{23}
 \end{aligned}$$

Only 3 multiplications needed instead of 5!

2.4 Modular Inverse

For decryption, we need to compute modular inverses.

[Fermat's Little Theorem] If p is prime and $\gcd(a, p) = 1$, then:

$$a^{p-1} \equiv 1 \pmod{p} \quad (3)$$

Therefore, the modular inverse is:

$$a^{-1} \equiv a^{p-2} \pmod{p} \quad (4)$$

Example: Finding $6^{-1} \bmod 23$

Using Fermat's Little Theorem:

$$6^{-1} \equiv 6^{23-2} \equiv 6^{21} \pmod{23}$$

Computing using fast exponentiation:

$$6^{21} \equiv 4 \pmod{23}$$

Verification: $6 \times 4 = 24 \equiv 1 \pmod{23}$ ✓

3 ElGamal Encryption Algorithm

3.1 Overview of the Three Phases

ElGamal encryption consists of three main phases:

1. **Key Generation:** Creating public and private key pairs
2. **Encryption:** Converting plaintext to ciphertext
3. **Decryption:** Recovering plaintext from ciphertext

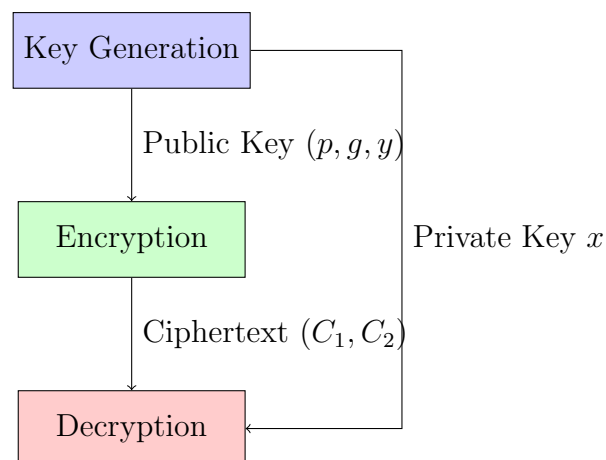


Figure 1: ElGamal Encryption Process Flow

3.2 Step 1: Key Generation

The recipient (Bob) generates a public-private key pair.

Key Generation Algorithm**Input:** Security parameter (bit length)**Steps:**

1. Choose a large prime number p (at least 2048 bits for security)
2. Choose a generator g (primitive root modulo p)
3. Choose private key x randomly where $1 < x < p - 1$
4. Compute public key: $y = g^x \bmod p$

Output:

- **Public Key:** (p, g, y) — published openly
- **Private Key:** x — kept secret

Example: Key Generation

Let's generate keys with small values for demonstration:

Step 1: Choose prime $p = 23$ **Step 2:** Choose generator $g = 5$ (we can verify 5 is a primitive root mod 23)**Step 3:** Choose private key $x = 6$ (randomly selected)**Step 4:** Compute public key:

$$\begin{aligned} y &= g^x \bmod p \\ &= 5^6 \bmod 23 \\ &= 15625 \bmod 23 \\ &= 8 \end{aligned}$$

Result:

- Public Key: $(23, 5, 8)$
- Private Key: 6

3.3 Step 2: Encryption Process

The sender (Alice) encrypts a message for Bob.

Encryption Algorithm

Input:

- Message M (as integer where $0 \leq M < p$)
- Bob's public key (p, g, y)

Steps:

1. Choose random k where $1 < k < p - 1$ (ephemeral key)
2. Compute $C_1 = g^k \bmod p$
3. Compute $C_2 = M \times y^k \bmod p$
4. Send ciphertext (C_1, C_2) to Bob

Output: Ciphertext pair (C_1, C_2)

Important Notes

- The random value k **must be different** for each encryption
- k should be generated using a cryptographically secure random number generator
- Never reuse k — this breaks security!
- The ciphertext is **twice the size** of the plaintext

Example: Encryption**Given:**

- Message: $M = 19$
- Public Key: $(p = 23, g = 5, y = 8)$
- Random value: $k = 3$

Step 1: Compute C_1

$$\begin{aligned}C_1 &= g^k \bmod p \\&= 5^3 \bmod 23 \\&= 125 \bmod 23 \\&= 10\end{aligned}$$

Step 2: Compute C_2

$$\begin{aligned}C_2 &= M \times y^k \bmod p \\&= 19 \times 8^3 \bmod 23 \\&= 19 \times 512 \bmod 23\end{aligned}$$

First, compute $512 \bmod 23$:

$$\begin{aligned}512 &= 22 \times 23 + 6 \\512 &\equiv 6 \pmod{23}\end{aligned}$$

Then:

$$\begin{aligned}C_2 &= 19 \times 6 \bmod 23 \\&= 114 \bmod 23 \\&= 22\end{aligned}$$

Result: Ciphertext = $(10, 22)$

3.4 Step 3: Decryption Process

Bob receives the ciphertext and decrypts using his private key.

Decryption Algorithm

Input:

- Ciphertext (C_1, C_2)
- Private key x

Steps:

1. Compute $s = C_1^x \bmod p$
2. Compute $s^{-1} = s^{p-2} \bmod p$ (using Fermat's Little Theorem)
3. Recover message: $M = C_2 \times s^{-1} \bmod p$

Output: Original message M

3.4.1 Why Decryption Works (Mathematical Proof)

Let's prove that decryption correctly recovers the message:

Proof. We need to show that $M = C_2 \times s^{-1} \bmod p$.

Step 1: Analyze s

$$\begin{aligned} s &= C_1^x \bmod p \\ &= (g^k)^x \bmod p \\ &= g^{kx} \bmod p \end{aligned}$$

Step 2: Analyze y^k

$$\begin{aligned} y^k &= (g^x)^k \bmod p \\ &= g^{xk} \bmod p \\ &= g^{kx} \bmod p \end{aligned}$$

Step 3: Therefore $s = y^k$

Step 4: Decrypt

$$\begin{aligned} C_2 \times s^{-1} &= (M \times y^k) \times (y^k)^{-1} \bmod p \\ &= M \times (y^k \times (y^k)^{-1}) \bmod p \\ &= M \times 1 \bmod p \\ &= M \bmod p \end{aligned}$$

Therefore, decryption correctly recovers the original message M . □ □

Example: Decryption**Given:**

- Ciphertext: $(C_1, C_2) = (10, 22)$
- Private Key: $x = 6$
- Prime: $p = 23$

Step 1: Compute $s = C_1^x \bmod p$

$$s = 10^6 \bmod 23$$

Using fast exponentiation:

$$10^2 = 100 \equiv 8 \pmod{23}$$

$$10^4 = (10^2)^2 = 8^2 = 64 \equiv 18 \pmod{23}$$

$$10^6 = 10^4 \times 10^2 = 18 \times 8 = 144 \equiv 6 \pmod{23}$$

So $s = 6$.**Step 2:** Compute $s^{-1} \bmod p$

$$\begin{aligned} s^{-1} &= 6^{-1} \bmod 23 \\ &= 6^{23-2} \bmod 23 \\ &= 6^{21} \bmod 23 \\ &= 4 \quad (\text{using fast exponentiation}) \end{aligned}$$

Verification: $6 \times 4 = 24 \equiv 1 \pmod{23}$ ✓**Step 3:** Recover message

$$\begin{aligned} M &= C_2 \times s^{-1} \bmod p \\ &= 22 \times 4 \bmod 23 \\ &= 88 \bmod 23 \\ &= 19 \end{aligned}$$

Result: Decrypted message = 19 ✓ (matches original!)

4 Complete Numerical Example

Let's work through a complete example from key generation to decryption.

Complete ElGamal Example

Scenario

Bob wants to receive encrypted messages. Alice wants to send him the message $M = 15$.

Phase 1: Key Generation (Bob)

Step 1: Choose prime $p = 29$

Step 2: Choose generator $g = 2$

Step 3: Choose private key $x = 12$ (random)

Step 4: Compute public key

$$y = g^x \bmod p = 2^{12} \bmod 29$$

Computing $2^{12} \bmod 29$:

$$2^1 = 2$$

$$2^2 = 4$$

$$2^4 = 16$$

$$2^8 = 256 \equiv 24 \pmod{29}$$

$$2^{12} = 2^8 \times 2^4 = 24 \times 16 = 384 \equiv 7 \pmod{29}$$

Bob publishes: Public Key = (29, 2, 7)

Bob keeps secret: Private Key = 12

Phase 2: Encryption (Alice)

Given: Message $M = 15$, Public Key (29, 2, 7)

Step 1: Choose random $k = 5$

Step 2: Compute C_1

$$C_1 = g^k \bmod p = 2^5 \bmod 29 = 32 \bmod 29 = 3$$

Step 3: Compute C_2

$$C_2 = M \times y^k \bmod p = 15 \times 7^5 \bmod 29$$

Computing $7^5 \bmod 29$:

$$7^1 = 7$$

$$7^2 = 49 \equiv 20 \pmod{29}$$

$$7^4 = 20^2 = 400 \equiv 23 \pmod{29}$$

$$7^5 = 7^4 \times 7 = 23 \times 7 = 161 \equiv 16 \pmod{29}$$

Then:

$$C_2 = 15 \times 16 \bmod 29 = 240 \bmod 29 = 8$$

Alice sends: Ciphertext = (3, 8)

Phase 3: Decryption (Bob)

Given: Ciphertext (3, 8), Private Key $x = 12$

5 Python Implementation

5.1 Complete ElGamal Implementation

Below is a complete, well-documented Python implementation of the ElGamal encryption system:

```

1 import random
2
3 def mod_exp(base, exp, mod):
4     """
5     Efficient modular exponentiation using binary method.
6     Computes (base^exp) mod mod efficiently.
7
8     Args:
9         base: Base number
10        exp: Exponent
11        mod: Modulus
12
13    Returns:
14        Result of (base^exp) mod mod
15
16    Time Complexity: O(log exp)
17    """
18    result = 1
19    base = base % mod
20
21    while exp > 0:
22        if exp % 2 == 1: # If exp is odd
23            result = (result * base) % mod
24        exp = exp >> 1 # Divide exp by 2 (right shift)
25        base = (base * base) % mod # Square the base
26
27    return result
28
29 def mod_inverse(a, p):
30     """
31     Compute modular inverse using Fermat's Little Theorem.
32     For prime p: a^(-1) = a^(p-2) (mod p)
33
34     Args:
35         a: Number to find inverse of
36         p: Prime modulus
37
38    Returns:
39        Modular inverse of a modulo p
40
41    Note: Only works when p is prime and gcd(a, p) = 1
42    """
43    return mod_exp(a, p - 2, p)
44
45 def is_primitive_root(g, p):
46     """
47     Check if g is a primitive root modulo p.
48     A primitive root generates all elements in Z*_p.
49
50     Args:
51         g: Potential generator

```

```

52     p: Prime modulus
53
54     Returns:
55         True if g is a primitive root, False otherwise
56     """
57     # Check if  $g^{(p-1)/q} \neq 1$  for all prime factors q of p-1
58     # Simplified check: generate all powers and check if we get p-1
    distinct values
59     if g <= 1 or g >= p:
60         return False
61
62     seen = set()
63     power = 1
64     for i in range(1, p):
65         power = (power * g) % p
66         if power in seen:
67             return False
68         seen.add(power)
69
70     return len(seen) == p - 1
71
72 class ElGamal:
73     """
74     ElGamal Encryption System Implementation
75
76     This class implements the ElGamal public-key cryptosystem
77     based on the discrete logarithm problem.
78     """
79
80     def __init__(self, p, g):
81         """
82         Initialize ElGamal with prime p and generator g.
83
84         Args:
85             p: Large prime number (modulus)
86             g: Generator (primitive root modulo p)
87
88         Raises:
89             ValueError: If p is not valid or g is not a generator
90         """
91         self.p = p # Large prime
92         self.g = g # Generator
93
94         # Validate inputs
95         if self.p < 3:
96             raise ValueError("Prime p must be at least 3")
97         if self.g < 2 or self.g >= self.p:
98             raise ValueError(f"Generator g must be between 2 and {self.p-1}")
99
100     def generate_keys(self):
101         """
102         Generate public and private key pair.
103
104         Returns:
105             tuple: (public_key, private_key)
106                 public_key: tuple (p, g, y)
107                 private_key: integer x

```

```

108     """
109     # Choose random private key x in range [2, p-2]
110     x = random.randint(2, self.p - 2)
111
112     # Compute public key y = g^x mod p
113     y = mod_exp(self.g, x, self.p)
114
115     # Return keys
116     public_key = (self.p, self.g, y)
117     private_key = x
118
119     return public_key, private_key
120
121 def encrypt(self, message, public_key):
122     """
123     Encrypt a message using ElGamal encryption.
124
125     Args:
126         message: Integer message (must satisfy 0 <= M < p)
127         public_key: tuple (p, g, y) - recipient's public key
128
129     Returns:
130         tuple: Ciphertext (C1, C2)
131
132     Note: Each encryption uses a fresh random value k,
133           so same message produces different ciphertexts.
134     """
135     p, g, y = public_key
136
137     # Validate message
138     if message < 0 or message >= p:
139         raise ValueError(f"Message must be in range [0, {p-1}]")
140
141     # Choose random ephemeral key k in range [2, p-2]
142     k = random.randint(2, p - 2)
143
144     # Compute C1 = g^k mod p
145     C1 = mod_exp(g, k, p)
146
147     # Compute C2 = M * y^k mod p
148     C2 = (message * mod_exp(y, k, p)) % p
149
150     return (C1, C2)
151
152 def decrypt(self, ciphertext, private_key):
153     """
154     Decrypt ciphertext using ElGamal decryption.
155
156     Args:
157         ciphertext: tuple (C1, C2) - encrypted message
158         private_key: integer x - recipient's private key
159
160     Returns:
161         int: Original message M
162     """
163     C1, C2 = ciphertext
164
165     # Compute s = C1^x mod p

```

```

166     s = mod_exp(C1, private_key, self.p)
167
168     # Compute s(-1) mod p using Fermat's Little Theorem
169     s_inv = mod_inverse(s, self.p)
170
171     # Recover message M = C2 * s(-1) mod p
172     message = (C2 * s_inv) % self.p
173
174     return message
175
176 def encrypt_string(self, text, public_key):
177     """
178     Encrypt a string message by converting to integers.
179
180     Args:
181         text: String message to encrypt
182         public_key: tuple (p, g, y)
183
184     Returns:
185         list: List of ciphertext pairs [(C1, C2), ...]
186     """
187     ciphertexts = []
188
189     for char in text:
190         # Convert character to ASCII value
191         ascii_val = ord(char)
192
193         # Encrypt the ASCII value
194         ciphertext = self.encrypt(ascii_val, public_key)
195         ciphertexts.append(ciphertext)
196
197     return ciphertexts
198
199 def decrypt_string(self, ciphertexts, private_key):
200     """
201     Decrypt a list of ciphertexts back to string.
202
203     Args:
204         ciphertexts: List of ciphertext pairs
205         private_key: integer x
206
207     Returns:
208         str: Decrypted message
209     """
210     message = ""
211
212     for ciphertext in ciphertexts:
213         # Decrypt to get ASCII value
214         ascii_val = self.decrypt(ciphertext, private_key)
215
216         # Convert ASCII to character
217         message += chr(ascii_val)
218
219     return message
220
221 # =====
222 # Demonstration and Testing
223 # =====

```

```
224
225 def demonstrate_elgamal():
226     """
227     Comprehensive demonstration of ElGamal encryption system.
228     """
229     print("=" * 60)
230     print("ELGAMAL ENCRYPTION SYSTEM DEMONSTRATION")
231     print("=" * 60)
232
233     # Initialize with small prime for demonstration
234     p = 23
235     g = 5
236     elgamal = ElGamal(p, g)
237
238     print(f"\nSystem Parameters:")
239     print(f"  Prime (p): {p}")
240     print(f"  Generator (g): {g}")
241
242     # Generate keys
243     print("\n" + "-" * 60)
244     print("KEY GENERATION")
245     print("-" * 60)
246
247     public_key, private_key = elgamal.generate_keys()
248     p_pub, g_pub, y_pub = public_key
249
250     print(f"Public Key:")
251     print(f"  p = {p_pub}")
252     print(f"  g = {g_pub}")
253     print(f"  y = {y_pub}")
254     print(f"\nPrivate Key:")
255     print(f"  x = {private_key}")
256
257     # Encrypt a message
258     print("\n" + "-" * 60)
259     print("ENCRYPTION")
260     print("-" * 60)
261
262     message = 19
263     print(f"Original Message: {message}")
264
265     ciphertext = elgamal.encrypt(message, public_key)
266     C1, C2 = ciphertext
267
268     print(f"Ciphertext:")
269     print(f"  C1 = {C1}")
270     print(f"  C2 = {C2}")
271
272     # Decrypt the message
273     print("\n" + "-" * 60)
274     print("DECRYPTION")
275     print("-" * 60)
276
277     decrypted = elgamal.decrypt(ciphertext, private_key)
278     print(f"Decrypted Message: {decrypted}")
279
280     # Verify
281     if message == decrypted:
```

```

282         print("\n    SUCCESS! Encryption/Decryption verified.")
283     else:
284         print("\n    ERROR! Messages don't match.")
285
286     # Demonstrate probabilistic encryption
287     print("\n" + "-" * 60)
288     print("PROBABILISTIC ENCRYPTION PROPERTY")
289     print("-" * 60)
290     print("Encrypting the same message multiple times:\n")
291
292     for i in range(1, 4):
293         ct = elgamal.encrypt(message, public_key)
294         print(f"    Encryption {i}: {ct}")
295
296     print("\nNote: Different ciphertexts for the same message!")
297     print("This is semantic security in action.")
298
299 # Run demonstration
300 if __name__ == "__main__":
301     demonstrate_elgamal()

```

5.2 Example Output

When you run the above program, you'll see output similar to:

```

=====
ELGAMAL ENCRYPTION SYSTEM DEMONSTRATION
=====

System Parameters:
  Prime (p): 23
  Generator (g): 5

-----
KEY GENERATION
-----

Public Key:
  p = 23
  g = 5
  y = 8

Private Key:
  x = 6

-----
ENCRYPTION
-----

Original Message: 19
Ciphertext:
  C1 = 10
  C2 = 22

-----
DECRYPTION
-----

Decrypted Message: 19

```

```
SUCCESS! Encryption/Decryption verified.

-----
PROBABILISTIC ENCRYPTION PROPERTY
-----
Encrypting the same message multiple times:

Encryption 1: (14, 15)
Encryption 2: (2, 7)
Encryption 3: (18, 11)

Note: Different ciphertexts for the same message!
This is semantic security in action.
```

6 Key Features of ElGamal

6.1 Advantages

Strengths of ElGamal

1. Semantic Security (Probabilistic Encryption)

- Same plaintext produces different ciphertexts
- Resistant to chosen-plaintext attacks
- Attacker gains no information about plaintext

2. Homomorphic Property

- Supports multiplicative homomorphism
- If $E(m_1) = (C_{1,1}, C_{1,2})$ and $E(m_2) = (C_{2,1}, C_{2,2})$
- Then $E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2)$
- Useful for certain cryptographic protocols

3. No Patent Restrictions

- Unlike RSA (patented until 2000)
- Free to use in any application
- Open implementation without licensing

4. Different Mathematical Foundation

- Based on discrete logarithm problem
- Diversifies cryptographic security
- Not affected by RSA-specific attacks

6.2 Disadvantages

Limitations of ElGamal

1. Ciphertext Expansion

- Ciphertext is $2\times$ size of plaintext
- (C_1, C_2) both need to be transmitted
- Increases bandwidth requirements
- Not ideal for large messages

2. Randomness Requirement

- Needs cryptographically secure random k for each encryption
- Poor RNG can compromise security
- Must never reuse k values

3. Performance

- Generally slower than RSA
- Requires two exponentiations per encryption
- Decryption also computationally expensive

4. Message Space Limitation

- Message must be $< p$
- Large messages need to be split
- Hybrid encryption often used in practice

6.3 Homomorphic Property Demonstration

ElGamal has a multiplicative homomorphic property:

[Multiplicative Homomorphism] Let $E(m_1) = (C_{1,1}, C_{1,2})$ and $E(m_2) = (C_{2,1}, C_{2,2})$ be encryptions of messages m_1 and m_2 . Then:

$$(C_{1,1} \cdot C_{2,1} \bmod p, C_{1,2} \cdot C_{2,2} \bmod p) = E(m_1 \cdot m_2 \bmod p) \quad (5)$$

Example: Homomorphic Property

Given $p = 23$, $g = 5$, $y = 8$, and messages $m_1 = 3$, $m_2 = 4$:

Encrypt $m_1 = 3$ **with** $k_1 = 2$:

$$C_{1,1} = 5^2 \bmod 23 = 2$$

$$C_{1,2} = 3 \times 8^2 \bmod 23 = 3 \times 18 = 54 \bmod 23 = 8$$

Encrypt $m_2 = 4$ **with** $k_2 = 3$:

$$C_{2,1} = 5^3 \bmod 23 = 10$$

$$C_{2,2} = 4 \times 8^3 \bmod 23 = 4 \times 6 = 24 \bmod 23 = 1$$

Multiply ciphertexts:

$$\begin{aligned} C_1 \cdot C_2 &= (2 \times 10 \bmod 23, 8 \times 1 \bmod 23) \\ &= (20, 8) \end{aligned}$$

Verify: This should decrypt to $m_1 \times m_2 = 3 \times 4 = 12 \bmod 23$

This property is useful in secure voting, sealed-bid auctions, and privacy-preserving protocols!

7 Security Considerations

7.1 Security Foundation

The security of ElGamal relies on the **computational Diffie-Hellman (CDH) assumption**:

Computational Diffie-Hellman Problem

Given: g , g^a , and g^b (all mod p)

Find: $g^{ab} \bmod p$

Assumption: This is computationally infeasible for properly chosen parameters

Relation to ElGamal: Breaking ElGamal is at least as hard as solving CDH

7.2 Critical Security Rules

Security Best Practices

1. **NEVER Reuse Random Value k**
 - Most critical security requirement
 - Reusing k completely breaks the system
 - See attack demonstration below
2. **Use Large Prime p**
 - Minimum 2048 bits for current security
 - 3072 bits recommended for long-term security
 - Small primes (like our examples) are for learning only!
3. **Secure Random Number Generation**
 - Use cryptographically secure RNG (e.g., `secrets` module in Python)
 - Never use predictable or pseudorandom generators
 - Insufficient entropy can be exploited
4. **Proper Generator Selection**
 - g should be a primitive root modulo p
 - Or use a generator of a large prime-order subgroup
 - Standard primes (like those in RFC 3526) recommended

7.3 Attack: Reusing Random Value k

This is the most devastating attack on ElGamal!

The Nonce Reuse Attack

Scenario: Alice encrypts two messages M_1 and M_2 using the same k value.

Ciphertexts:

$$E(M_1) = (C_1, C_{2,1}) = (g^k \bmod p, M_1 \times y^k \bmod p)$$

$$E(M_2) = (C_1, C_{2,2}) = (g^k \bmod p, M_2 \times y^k \bmod p)$$

Observation: C_1 is the same for both!

Attack:

$$\frac{C_{2,1}}{C_{2,2}} = \frac{M_1 \times y^k}{M_2 \times y^k} = \frac{M_1}{M_2} \bmod p$$

If attacker knows M_1 (or can guess it), they can compute:

$$M_2 = M_1 \times (C_{2,1})^{-1} \times C_{2,2} \bmod p \quad (6)$$

Lesson: ALWAYS use a fresh random k for each encryption!

Example: Nonce Reuse Attack

Setup: $p = 23, g = 5, y = 8, k = 3$ (same for both encryptions)

Encrypt $M_1 = 10$:

$$C_1 = 5^3 = 10 \bmod 23$$

$$C_{2,1} = 10 \times 8^3 = 10 \times 6 = 60 \bmod 23 = 14$$

Encrypt $M_2 = 15$ (using same k):

$$C_1 = 5^3 = 10 \bmod 23 \quad (\text{same!})$$

$$C_{2,2} = 15 \times 8^3 = 15 \times 6 = 90 \bmod 23 = 21$$

Attack (assume attacker knows $M_1 = 10$):

$$\begin{aligned} \frac{C_{2,2}}{C_{2,1}} &= \frac{21}{14} \bmod 23 \\ &= 21 \times 14^{-1} \bmod 23 \\ &= 21 \times 5 \bmod 23 \quad (\text{since } 14^{-1} = 5 \bmod 23) \\ &= 105 \bmod 23 = 13 \end{aligned}$$

Then:

$$M_2 = M_1 \times \frac{C_{2,2}}{C_{2,1}} = 10 \times 13 = 130 \bmod 23 = 15$$

Success! Attacker recovered M_2 without the private key! ✓

7.4 Chosen Ciphertext Attacks

ElGamal in its basic form is **malleable**, meaning:

- An attacker can modify ciphertexts to produce related plaintexts
- Not secure against chosen ciphertext attacks (CCA)
- Solution: Use padding schemes like OAEP (Optimal Asymmetric Encryption Padding)

8 Comparison: ElGamal vs RSA

8.1 Comprehensive Comparison Table

Table 1: ElGamal vs RSA Comparison

Feature	ElGamal	RSA
Security Basis	Discrete Logarithm Problem (DLP)	Integer Factorization Problem
Encryption Type	Probabilistic (semantic security)	Deterministic (without padding)
Ciphertext Size	$2 \times$ plaintext size	Same as plaintext
Randomization	Required for every encryption	Optional (with padding)
Same Plaintext	Different ciphertexts	Same ciphertext
Speed (Encryption)	Slower (2 exponentiations)	Moderate (1 exponentiation)
Speed (Decryption)	Moderate	Faster (with CRT)
Patent Status	Never patented (free)	Patented until 2000
Homomorphic	Multiplicative	Multiplicative
CCA Security	Not inherently secure	Not inherently secure
Key Generation	Moderate	Slow (need two primes)
Bandwidth	Higher (2 values)	Lower (1 value)
Malleability	Malleable	Malleable

8.2 Time Complexity Analysis

Table 2: Computational Complexity Comparison

Operation	ElGamal	RSA
Key Generation	$O(\log^2 p)$	$O(\log^3 n)$
Encryption	$2 \times O(\log^2 p)$	$O(\log^2 n)$
Decryption	$O(\log^2 p)$	$O(\log^2 n)$ with CRT

Note: Complexity is per modular exponentiation operation.

8.3 When to Use Each

Decision Guide

Use ElGamal when:

- Semantic security is required
- Probabilistic encryption is desired
- Patent-free solution needed (historical concern)
- Working with discrete-log based protocols
- Bandwidth is not a primary concern

Use RSA when:

- Deterministic encryption is acceptable (with proper padding)
- Bandwidth is limited (smaller ciphertexts)
- Faster decryption is priority
- Standard PKI infrastructure required
- Digital signatures needed (RSA signatures are standard)

Modern Practice:

- Use **hybrid encryption** for large messages
- ElGamal/RSA for key encapsulation
- AES for actual data encryption
- Combine with proper padding schemes (OAEP, PSS)

9 Practical Applications

9.1 Real-World Usage

9.1.1 1. Digital Signatures (ElGamal Signature Scheme)

ElGamal can be adapted for digital signatures:

- **DSA (Digital Signature Algorithm)**: Based on ElGamal signatures
- **Schnorr Signatures**: Variant of ElGamal signatures
- Used in PGP, SSH, and various authentication protocols

9.1.2 2. Hybrid Encryption Systems

Common Pattern:

1. Generate random symmetric key K (e.g., AES key)
2. Encrypt K using ElGamal: $C_{ElGamal} = E_{ElGamal}(K)$
3. Encrypt message using K : $C_{data} = E_{AES}(M, K)$
4. Send $(C_{ElGamal}, C_{data})$

This combines:

- Security of ElGamal for key exchange
- Speed of symmetric encryption for data
- Avoids ciphertext expansion problem

9.1.3 3. Secure Voting Systems

ElGamal's homomorphic property enables:

- Encrypted votes can be tallied without decryption
- Individual vote privacy maintained
- Verifiable results without revealing votes

9.1.4 4. Sealed-Bid Auctions

- Bids encrypted with ElGamal
- Homomorphic property allows comparison
- Winner determined without revealing losing bids

9.2 Implementation in Python (Production-Ready)

For real-world applications, use established libraries:

```
1 # Using PyCryptodome library
2 from Crypto.PublicKey import ElGamal
3 from Crypto.Random import get_random_bytes
4 from Crypto.Hash import SHA256
5
6 # Generate keys (2048-bit security)
7 key = ElGamal.generate(2048, get_random_bytes)
8
9 # Export public key
10 public_key = key.publickey().exportKey()
11
12 # Encrypt message
13 message = b"Secret message"
14 k = get_random_bytes(32) # Random value
15 ciphertext = key.encrypt(message, k)
16
17 # Decrypt
18 plaintext = key.decrypt(ciphertext)
19
20 print(f"Original: {message}")
21 print(f"Decrypted: {plaintext}")
```

Important

Never implement cryptographic primitives from scratch for production!
Use audited libraries:

- **Python:** PyCryptodome, cryptography
- **Java:** Bouncy Castle
- **C/C++:** OpenSSL, libsodium
- **JavaScript:** crypto (Node.js), Web Crypto API

Our implementations are for **educational purposes only!**

10 Summary and Key Takeaways

10.1 Core Concepts Review

Essential Points to Remember

1. **ElGamal is a public-key cryptosystem** based on the discrete logarithm problem
2. **Three phases:**
 - Key Generation: Choose p, g, x ; compute $y = g^x \bmod p$
 - Encryption: $(C_1, C_2) = (g^k \bmod p, M \times y^k \bmod p)$
 - Decryption: $M = C_2 \times (C_1^x)^{-1} \bmod p$
3. **Random value k must be:**
 - Different for each encryption
 - Cryptographically secure random
 - NEVER reused (catastrophic if violated)
4. **Security relies on:**
 - Difficulty of computing discrete logarithms
 - Computational Diffie-Hellman assumption
 - Proper parameter selection
5. **Key property:** Probabilistic encryption (semantic security)
 - Same message $\rightarrow$ different ciphertexts
 - Protects against many attacks

10.2 Mathematical Formulas Summary

Quick Reference Formulas

Key Generation:

$$x \in_R [2, p-2] \quad (\text{private key})$$

$$y = g^x \bmod p \quad (\text{public key component})$$

Encryption (message M , random k):

$$C_1 = g^k \bmod p$$

$$C_2 = M \times y^k \bmod p$$

$$\text{Ciphertext} = (C_1, C_2)$$

Decryption (private key x):

$$s = C_1^x \bmod p$$

$$s^{-1} = s^{p-2} \bmod p \quad (\text{Fermat's Little Theorem})$$

$$M = C_2 \times s^{-1} \bmod p$$

Why it works:

$$s = C_1^x = (g^k)^x = g^{kx} = (g^x)^k = y^k \bmod p$$

10.3 Security Principles

- ✓ Always use large primes (2048+ bits)
- ✓ Use cryptographically secure RNG
- ✓ Never reuse k values
- ✓ Validate all inputs
- ✓ Use proper padding schemes (OAEP)
- × Don't use small primes (like examples)
- × Don't implement crypto from scratch
- × Don't use predictable random values

11 Practice Questions

11.1 Question 1: Basic Calculation

Problem: Given $p = 11$, $g = 2$, and private key $x = 3$:

- (a) Calculate the public key y

- (b) Encrypt message $M = 5$ using random $k = 4$
- (c) Decrypt the ciphertext to verify

Show all intermediate steps and calculations.

11.2 Question 2: Implementation Challenge

Problem: Write a Python function that:

- (a) Takes a string message as input
- (b) Converts each character to its ASCII value
- (c) Encrypts each character using ElGamal
- (d) Returns a list of ciphertext pairs
- (e) Implements a corresponding decryption function

Test your implementation with the message "HELLO" and verify decryption.

11.3 Question 3: Security Analysis

Problem: Explain why using the same value of k for encrypting two different messages is a security vulnerability.

- (a) Provide a mathematical proof showing how an attacker can exploit this
- (b) Demonstrate with a numerical example
- (c) Explain what information the attacker can extract
- (d) Suggest how to prevent this attack in practice